IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

Low Level Design

on

Identity Security Framework For Agriculture

Student Name: Aparna A.N

Industry Mentor:
Mr. Shashank

Guided by: Mr. Vikyath Shetty

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview. 4

2. System Design. 4
   2.1. Application Design 4
   2.2. Process Flow. 5
   2.3. Information Flow. 5
   2.4. Components Design 6
   2.5. Key Design Considerations 6
   2.6. API Catalogue. 6

3. Data Design. 7
   3.1. Data Model 7
   3.2. Data Access Mechanism 7
   3.3. 3.3. Data Retention Policies 7
   3.4. Data Migration 7

4. Interfaces 8

5. State and Session Management 8

6. Caching 8

7. Non-Functional Requirements 8
   7.1. Security Aspects 8
   7.2. Performance Aspects 9

8. References 9

1. Introduction

This Low-Level Design document provides a detailed technical blueprint for the **Identity Security Framework for Agriculture**. It serves as the primary reference for developers to transform high-level architectural requirements into functional, secure code. The document specifies the granular details of database entities, internal class structures, API endpoint logic, and validation rules required to maintain a robust identity ecosystem for agricultural stakeholders.

1.1 Scope of the Document

- Identity Management: Detailed schema for the User entity, including role-based attributes and security flags.
- Security & Authentication: Specification of password hashing algorithms, session handling, and server-side validation logic.
- Authorization : Implementation details for Role-Based Access Control distinguishing "Admin" and "Farmer" privileges.
- Data Persistence: SQL schemas for both user profiles and agricultural transaction (Order) logs.
- Administrative Logic: Internal workflows for account deactivation, user deletion, and order status transitions (Approved/Rejected/Pending).

1.2 Intended Audience

- Full-Stack Developers: For implementing the backend routing and frontend template integration.
- Database Administrators: To ensure the SQLite schema adheres to the defined integrity constraints.

1.3 System Overview

The framework is a security-first platform designed to digitize trust in agricultural commercialization. It allows farmers to register and trade crops while providing administrators with an elevated control plane to manage user lifecycles and monitor global sales.

2. System Design

2.1 Application Design

```
[ CLIENT LAYER]
  └── User / Admin Browser
        |
        ▼
[ SERVER LAYER (Flask)]
 |
 ├── Auth Module (Security & Sessions)
 ├── Admin Module (User & Order)
 ├── Market Module (Crop Inventory)
 └── Notification Module (User Alerts)
       |
       ▼
[ DATA LAYER (SQLite) ]
  └── database.db (Persistent Storage)
```

2.2 Process Flow

- Signup Process
  - Enter Details: User fills in their Name, Email, and Password.
  - Validation: The system checks if the email is typed correctly and isn't already used by someone else.
  - Security: The system "locks" the password using secure encryption so it can't be read in plain text.
  - Save and Finish: The user's information is saved safely in the database and sent to the Login page to start using the app.
- Login Process
  - Enter Credentials: User enters their Email and Password.
  - Identity Check: System looks for the user's email and checks if the password matches the one stored.
  - Account Check: The system makes sure the account hasn't been disabled by an admin.
  - Start Session: If everything is correct, the system logs the user in and remembers their visit.

2.3 Information Flow

- Request Phase: HTTP POST carries form data (ID, credentials).
- Processing Phase: Flask extracts data ,Validates types and Executes SQL.
- Persistence Phase: SQLite writes to database.db root file.
- Feedback Phase: Flash messages are queued into the session Next page load renders alerts for the user.

2.4 Component Design

- Auth Module: Responsible for secure signup, login verification, and session management using Flask-Login.
- Marketplace Module: Manages the crop inventory display and the purchasing logic.
- Admin Module: Provides elevated access to modify user accounts and monitor global sales.
- Notification Module: Handles the creation and display of real-time alerts for farmers based on admin actions.
- Database Module: Uses the sqlite3 library with a Row Factory pattern for direct, efficient database interaction.

2.5 Key Design consideration

- Security: PBKDF2 hashing and 100% parameterized SQL to prevent injections.
- Account Kill-Switch: Admins can deactivate users instantly via the is_active_account flag.
- Automated Alerts: Real-time notifications for farmers when order statuses change.
- RBAC: Strict server-side separation of Admin and Farmer privileges.
- Lightweight: Minimal dependencies (Flask + SQLite) for fast, low-resource operation.

2.5 API Catalogue

Endpoint	Method	Description	Access
`/login`	GET, POST	User authentication & status check.	Public
`/signup`	GET, POST	Registration with Admin Code validation.	Public
`/crops`	GET	Marketplace catalog for farmers.	Farmer
`/buy`	POST	Order placement logic.	Farmer

Endpoint	Method	Description	Access
`/admin`	GET	User management portal.	Admin
`/admin/sales`	GET	Sales and order monitoring.	Admin
`/admin/toggle_status/<id>`	GET	Deactivate/Activate user accounts.	Admin
`/admin/order/<id>/<act>`	GET	Approve or Reject farmer orders.	Admin
`/mark_read/<id>`	GET	Dismiss user notifications.	Farmer

3. Data Design

3.1 Data Model

- user: Stores id, name, email, encrypted password_hash, role, and activity status.
- order: Stores id, user_id (link to user), crop_name, quantity, price, shipping address, and order status.

3.2 Data Access Mechanism

The application uses the built-in Python sqlite3 library. It implements a Row Factory pattern, allowing software components to interact with database records using name-based lookups (e.g., row['email']), ensuring the code is stable and easy to maintain.

3.3 Data Retention Policies

All information is stored in a persistent local file (database.db) in the project root. Data is retained until an administrator explicitly performs a "Delete" action, which triggers a permanent removal of the record from the SQL file.

3.4 Data Migration

The system uses a direct initialization script (fix_db.py) to handle database creation and schema resets.This ensures that every developer or environment starts with a perfectly identical database structure in the root directory.

4. Interfaces

- Frontend Interface: A browser-based web UI using HTML5 and CSS for cross-device compatibility.
- Database Interface: Direct filesystem I/O between the Flask app and the SQLite database file.
- Session Interface: Uses Secure Signed Cookies to maintain user state. Flask-Login acts as the interface between the web request and the serialized User object.

5. State And Session Management

The system uses secure, signed Cookies to manage user sessions. By using a Secret Key, the application can verify that the session has not been modified by the user. Flask-Login is used to track the "current_user" across different sections of the app.

6. Caching

- Browser Caching: Static CSS and images are signaled for browser-side caching to speed up user experience.
- SQL Caching: The system relies on the internal SQLite and Operating System file buffers to quickly retrieve frequently accessed data like the crop marketplace list.

7. NON- Functional Requirements

7.1 Security Aspects

- Cryptography: All passwords are hashed with high-iteration PBKDF2 encryption.
- Role-Based Access: Admin features are physically restricted by server-side logic; a standard user cannot access them even if they know the URL.
- SQL Safety: Parameterized queries are used for every database action to prevent SQL Injection attacks.

7.2 Performance Aspects

- High Speed: By using a local SQLite file in the root, database lookups usually take less than 1 millisecond.
- Lightweight: The application has a very low memory footprint, making it suitable for standard web servers or local agricultural management hardware.

8. References

- Flask Documentation: https://flask.palletsprojects.com/
- Python sqlite3 Documentation: https://docs.python.org/3/library/sqlite3.html